

Graph-Based AI for Job-Skill Recommendation: Bridging the Talent Gap with Advanced Graph Neural Networks

Kshitij Desai
University of Adelaide
kshitij.desai@student.adelaide.edu.au

November 2024

Abstract

The objective of this project is to recommend suitable jobs based on user skills and preferences. Various machine Learning methods were explored – Matrix factorization and graph neural network. Models are trained and tested on Stack overflow dataset. This dataset includes latest information about user skills and their job details. Two different types of graph neural network based approach were investigated – Homogeneous and Heterogeneous. The results were evaluated using Mean Reciprocal Rank (MRR). It was found that choice of model affects recommendation quality significantly.

Keywords

GCNN, GraphSAGE, Inductive Learning, Meta-paths, Attention Mechanism

1 INTRODUCTION

Both hiring and searching for a job are exhaustive process because most of the times this process lacks personalization and is very robotic. Jobs seekers are recommended all sort of jobs weather it aligns with their personal preferences or not. Having an overlap of skills that a job role requires and the job seeker has is not enough for recommendation. Hiring systems need to have a mechanism in which they can match job seekers with jobs based on global characteristics like what roles other candidates with similar profile are working in, for same set of skills which role aligns more closely with the job seeker aspirations. It is also important to recommend relevant jobs at the top of the list, as candidates

can only apply to limited number of jobs in a duration of time. This touch of personalization can be achieved through building machine learning model that learns both job seekers and hiring manager’s behavior and addresses the unique behavior of both. To achieve this, I have shortlisted 3 algorithms using graph-based approach. The reason I chose this approach was due to the fact that most traditional machine learning models assume data belongs to same distribution and independent of each other. Graph based models capture the interconnected nature of user profiles and job requirements. We can either use a Matrix factorization-based technique which is widely used in industry to recommend products to users based on similar users in the past or we can use graph neural network which model our data as a series of nodes and edges. Through information propagation and aggregation mechanism, we can allow every node to learn the global information present in the graph without data leak by masking few nodes as test nodes and avoiding data propagation through them. Through this project I want to illustrate the potential of graph-based techniques in transforming job recommendation systems by creating a context-aware, dynamic, and personalized experience for job seekers.

2 RELATED WORKS

Recommendation algorithms are widely used across industries, from providing product recommendations to movie recommendations. This analogy can be extended to recommending jobs. Broadly, these recommendation algorithms can be divided into two categories: traditional, which involves techniques like user- and item-based collaborative filtering, and modern, which involves techniques using graph neural networks. Recommending jobs involves processing and filtering all the data shared by users, so that we only provide signal to the model devoid of noise [3, 4, 5, 6]. Liu et al. used the Multi-Graph Neural Network-based Skill Prediction model (MGNSP) in data-driven job skill prediction by using a multi-graph learning model that processes three different graphs: job-skill, skill-skill, and job-job graphs, textual feature learning to extract semantic information from job descriptions, and a fusion and prediction layer to make the final prediction [8].

David Topps et al. explored heterogeneous graphs and found that these graphs, when used in healthcare, can trace patterns, optimize pathways, and even personalize decisions, leading to more accurate recommendations [7]. Ping Liu et al. built LinkSAGE, an encoder-decoder-based GNN architecture to address the complexities of job matchmaking in the extremely large professional network at LinkedIn. They found significant improvement in relevance quality and hiring outcomes [2]. Rex Ying et al. proposed PinSage, which uses a Graph Convolution Network to generate high-quality recommendations compared to other deep learning and graph-based alternatives [1]. Uday Jain et al. developed an interactive dashboard that allows job seekers to explore job recommendations in a personalized way based on their skills and competency [9]. Amber Nigam et al. used sequence modeling with attention to capture nuances in how candidates

select jobs over time [10].

3 METHODOLOGY

Given the real-world nature of the data, preprocessing was critical before model building. Initial exploratory data analysis (EDA) highlighted relevant features like user skills and role types. Data was then split into training (90%) and testing (10%) sets to ensure robust evaluation.

3.1 Data pre-processing

After selecting the relevant rows and columns for training, columns which appeared similar like the languages, tools, databases and web frames the candidate has worked with, were combined to form 1 unified feature to be processed. All rows which had empty rows for labels were removed and label encoder was used to encode categorical variables. All the skills which were combined before were separated into multiple rows before encoding them.

3.2 Model Approaches

Three distinct modeling techniques were applied to analyses the relationship between user features and job roles:

3.2.1 Matrix Factorization Method

Algorithm 1 User-Based Collaborative Filtering

```

1: Input:
2:   Target user  $u$ 
3:   Target item  $i$ 
4:   Set of items rated by user  $u$ ,  $S_u$ 
5:   Set of users  $N(i)$  who have rated item  $i$ 
6:   User preference vectors  $\{p_v | v \in N(i)\}$ 
7:   Similarity weights  $\{w_{u,v} | v \in N(i)\}$ 
8: Output: Predicted score predict_score for user  $u$  on item  $i$ 
9:  $n_i \leftarrow \alpha(\{w_{u,v} \cdot p_v[i] \mid v \in N(i)\})$   $\triangleright$  Aggregate neighboring user preferences
   for item  $i$ 
10: predict_score  $\leftarrow \gamma(\{w_{u,v} \cdot n_i \mid v \in S_u\})$   $\triangleright$  Combine with user  $u$ 's
   preferences to generate final prediction
11: return predict_score

```

Explanation:

This pseudo code describes a way to predict how much a user might like an item by looking at the opinions of similar users. First, it finds users who have rated the item and combines their ratings, giving more weight to ratings from

users who are more similar to the target user. Then, it considers how much the target user has in common with these other users across all the items they've rated, creating a final predicted rating. This method works by assuming that users who liked or disliked similar items in the past will have similar tastes in the future.

3.2.2 Homogeneous Graph Neural Network

Algorithm 2 Graph Attention Network

```

1: Input:
2:   Current embedding  $h_u$  for node  $u$ 
3:   Set of neighboring embeddings  $\{h_v \mid v \in \mathcal{N}(u)\}$ 
4:   Weight matrix  $W$ 
5:   Attention vector  $a$ 
6:   Activation function  $\alpha$ 
7:   Softmax function for normalization
8: Output: New embedding  $h_u^{new}$  for node  $u$ 
9: Step 1: Transform node features:
10:   $h'_u \leftarrow W \cdot h_u$ 
11:   $h'_v \leftarrow W \cdot h_v \ \forall v \in \mathcal{N}(u)$ 
12: Step 2: Compute attention scores for each neighbor  $v \in \mathcal{N}(u)$ :
13:   $e_{u,v} \leftarrow \alpha(a^T[h'_u \parallel h'_v])$   $\triangleright$  Compute raw attention score using
    concatenation of transformed features
14: Step 3: Normalize attention scores across all neighbors of  $u$ :
15:   $\alpha_{u,v} \leftarrow \text{softmax}_v(e_{u,v})$   $\triangleright$  Apply softmax over all neighbors of  $u$  to get
    normalized attention coefficients
16: Step 4: Aggregate neighbor features based on attention scores:
17:   $n_u \leftarrow \gamma(\{\alpha_{u,v} \cdot h'_v \mid v \in \mathcal{N}(u)\})$   $\triangleright$  Weighted sum of neighbors
    embeddings using attention scores
18: Step 5: Compute final node embedding for  $u$ :
19:   $h_u^{new} \leftarrow \text{ReLU}(n_u + h'_u)$   $\triangleright$  Apply non-linear activation to combine  $u$ 's
    transformed feature with aggregated neighbors features
20: return  $h_u^{new}$ 

```

Explanation:

This pseudo code describes a Graph Attention Network (GAT) algorithm, which updates a node's representation by giving more importance to its most relevant neighbors. It first transforms each node's information, then computes attention scores that highlight important neighbors based on similarity. These scores are normalized, so the model knows which neighbors matter most. Finally, it combines this information, applies an activation to add a non-linear effect, and creates a new, updated representation for the node, emphasizing key connections.

3.2.3 Heterogeneous Graph Neural Network

Algorithm 3 HGNN Convolution Algorithm

```

1: Input:
2:   Current embedding  $z_u$  for node  $u$ 
3:   Set of neighbor's embeddings  $\{z_v \mid v \in \mathcal{N}(u)\}$ 
4:   Set of neighbor's weights
5:   Symmetric vector function  $\gamma(\cdot)$ 
6: Output: Updated node embedding  $z_u^{NEW}$ 
7: Step 1: Aggregate neighbor embeddings:
8:    $n_u \leftarrow \gamma(\{\text{ReLU}(Qh_v + q) \mid v \in \mathcal{N}(u)\}; \alpha)$   $\triangleright$  Aggregate transformed
      embeddings of neighbors using function  $\gamma$  and weights
9: Step 2: Combine and update node embedding:
10:   $z_u^{NEW} \leftarrow \text{ReLU}(W \cdot \text{concat}(z_u, n_u) + w)$   $\triangleright$ 
      Concatenate  $z_u$  with the aggregated neighbor embeddings  $n_u$ , apply linear
      transformation and ReLU activation
11: Step 3: Normalize the new embedding:
12:   $z_u^{NEW} \leftarrow \frac{z_u^{NEW}}{\|z_u^{NEW}\|_2}$   $\triangleright$  Normalize the new embedding
13: return  $z_u^{NEW}$ 

```

Explanation:

This pseudo code describes the Heterogeneous Graph Neural Network (HGNN) algorithm which updates a node's embedding by combining information from its neighbors. First, it aggregates the transformed embeddings of neighboring nodes using a function that weights them and applies an activation function. Next, it updates the target node by combining its own embedding with the aggregated neighbors' embeddings, applying another transformation and activation. Finally, the updated embedding is normalized to ensure consistency in its size, producing a refined representation that incorporates the most relevant neighbor's information.

4 Technical Novelty

This project introduces a novel use of Heterogeneous Graph Neural Networks (HGNNs) for personalized job recommendations, capturing the complex relationships between users, skills, and job roles. Unlike traditional models that treat data as independent and identically distributed, this approach models users, skills, and roles as unique nodes connected through meaningful relationships like “has_skill” and “requires_skill.” By employing HGNNs, the model learns these intricate patterns, allowing it to make recommendations that reflect not only current user skills but also align with broader career trajectories seen across similar profiles.

Moreover, by comparing three distinct methods—Matrix Factorization, Homogeneous GNN, and Heterogeneous GNN—this project highlights the advan-

tage of graph-based approaches over traditional recommendation techniques. The Heterogeneous GNN, in particular, demonstrates superior accuracy by leveraging its multi-relational structure, as seen in improved Mean Reciprocal Rank (MRR) scores. This shows that graph-based models, especially those that differentiate between node types, offer a more personalized and context-aware solution to job recommendations, bridging the gap between candidates' skills and the evolving demands of the job market.

5 EXPERIMENTS

5.1 Dataset

Stack Overflow 2024 dataset was used for this project featuring over 65,000 developers, provides a comprehensive look at skills, tools, frameworks, and job roles worldwide, serving as an ideal dataset for this study.

5.2 Evaluation Metric

The evaluation metric used to measure the quality of job recommendations in this project is the Mean Reciprocal Rank (MRR). MRR is calculated as follows:

$$\text{MRR} = \frac{1}{I} \sum_{i=1}^I \frac{1}{\text{rank}(i)}$$

Where:

- I is the total number of job-seekers.
- $\text{rank}(i)$ refers to the rank of the first relevant job in the recommendation list for user i .

Mean Reciprocal Rank (MRR) is the main evaluation metric for this project, used to evaluate the quality of ranking of the recommended jobs. MRR gives a strong hint about the rate at which relevant jobs are found by measuring the inverse rank of the first job in the recommendation list that is relevant. A larger MRR value shows that the system does better in earlier relevant job recommendations within the list.

5.3 Loss Metric: Cross-Entropy Loss

Cross-entropy loss is a standard metric used in classification tasks, which measures the difference between the true labels and the predicted probability distribution of the classes. Mathematically, it is defined as:

$$\text{Cross-Entropy} = - \sum_{c=1}^C y_c \cdot \log(p(y_c | x))$$

Where:

- C is the number of classes.
- y_c is a binary indicator (0 or 1) representing whether class c is the correct class.
- $p(y_c | x)$ is the predicted probability of class c for input x .

Cross-entropy loss penalizes high-confidence incorrect predictions, encouraging the model to output more accurate, probability-based recommendations. This aligns well with the goal of accurately classifying relevant jobs to improve ranking metrics such as MRR.

5.4 Optimizer: Adam with Learning Rate 0.001

Adam (Adaptive Moment Estimation) is an optimization algorithm that adaptively adjusts learning rates using moving averages of gradients and squared. Adam is chosen here for its adaptive nature and stability, allowing efficient convergence even in high-dimensional spaces typical in GNNs. The learning rate of 0.001 balances convergence speed and stability, making it ideal for achieving optimal performance on a large dataset like Stack Overflow.

5.5 Implementation Details

All the graph based neural network algorithms were implemented using PyTorch geometric using T4 Tesla GPU on Google Colab.

5.5.1 User-Based Collaborative Filtering

This approach leverages matrix factorization to model the relationship between users and job types, as outlined below:

- **Skill Aggregation:** Technical skills from various columns (e.g., languages, tools, frameworks) are combined to create a comprehensive skill profile for each user.
- **Job Profiles:** For each developer type (e.g., Data Scientist, Web Developer), a profile of top skills is generated based on the most frequently associated skills.
- **Similarity Calculation:** The model calculates skill overlap between users and job profiles using Jaccard Similarity, which measures the intersection of skills between the user and the job profile.
- **Interaction Matrix:** The similarity scores are organized into an interaction matrix, with users as rows and job types as columns, to represent the relationship between users and job profiles.
- **SVD for Dimensionality Reduction:** Singular Value Decomposition (SVD) is applied to reduce the interaction matrix into lower-dimensional latent factors, facilitating efficient prediction of job recommendations.

- **Job Recommendation:** For each user, the top job recommendations are generated based on the reconstructed preference scores derived from the latent factors.

5.5.2 GAT approach

This approach represents users, skills, and job types as nodes in a homogeneous graph, where edges signify relationships between them. The process is described below:

- **Data Preparation:** Key columns are retained, and user skills are aggregated into comprehensive skill profiles.
- **Node and Edge Creation:** Users, skills, and job roles are represented as nodes, with edges representing relationships between users and their skills, as well as between users and job roles.
- **GNN Architecture:** A Graph Attention Network (GATConv)-based architecture with 8 attention heads and 2 convolution layers is used to learn and model the relationships between nodes.
- **Training & Evaluation:** The model is trained for 50 epochs, with periodic accuracy checks to monitor performance.

5.5.3 HGNN approach

This approach uses a heterogeneous graph with distinct node types (users, skills, developer types) to model more complex relationships. The steps are outlined below:

- **Data Preparation:** Similar to previous methods, but in this case, nodes are distinguished by their type: user, skill, and developer type (dev_type).
- **Heterogeneous Graph Structure:** Connections are made based on specific relationships, such as "has_skill," "requires_skill," and "has_type."
- **HGNN Model Definition:** Using HeteroConv layers and SAGEConv, the model propagates messages across different node types, learning intricate connections and dependencies.
- **Training & Evaluation:** The model is trained for 50 epochs, with periodic accuracy checks to monitor performance.
- **Personalized Recommendations:** The model leverages user interactions to make personalized job suggestions via message passing.

6 Results

Method	Test Accuracy	MRR Score (Test Set)
Matrix Factorization	-	0.26
Homogeneous GNN	0.32	0.49
Heterogeneous GNN	0.52	0.58

Table 1: Comparison of recommendation quality using different methods. The Heterogeneous GNN achieves the highest accuracy and MRR score, highlighting its effectiveness in capturing complex, multi-typed relationships.

6.1 Plots

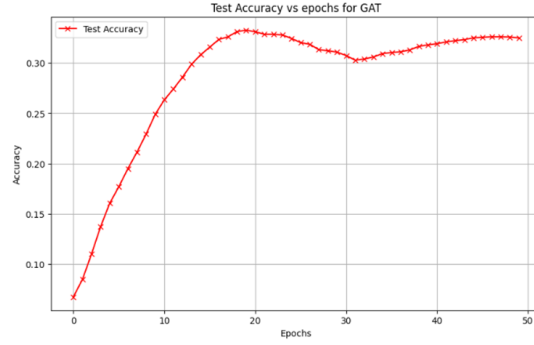


Figure 1: Testing characteristics of the GAT model.

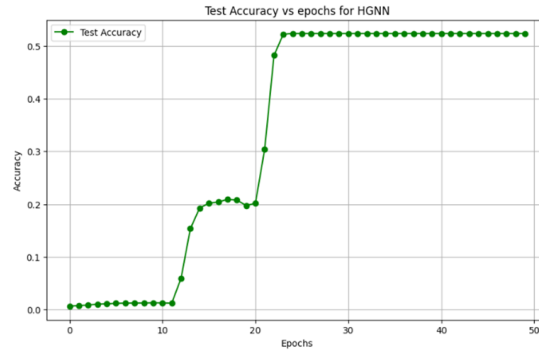


Figure 2: Testing characteristics of the HGNN model.

7 Technical Challenges and Resolution Strategy

Throughout this project, I encountered several technical challenges that required thoughtful problem-solving and refinement of my approach.

7.1 Data Leakage

Initially, I observed an unusually high Mean Reciprocal Rank (MRR) score of 0.95, which I believed was due to the HGNN model’s strong inductive learning capabilities. However, upon further analysis, I discovered that the model was predicting the same categories across most instances, indicating possible data leakage. This issue arose from the complex nature of data preparation for the HGNN and my initial setup. To resolve this, I implemented a test mask explicitly to hide the test data during training, ensuring a clearer separation between training and evaluation sets and improving the model’s generalization.

7.2 Data Preparation

Converting relational data from the original dataset into a graph structure was a non-trivial task, especially with the presence of categorical data and missing values. I employed techniques to encode categorical variables and handled missing data systematically, making it possible to represent the relational data in a graph format compatible with the HGNN model.

7.3 Memory Constraints on GPU

The model involved a substantial number of parameters, creating challenges in fitting the data onto a smaller GPU. I addressed this by optimizing model parameters, such as reducing the batch size and the number of attention heads. These adjustments helped manage memory usage without significantly compromising model performance.

8 Conclusion

In conclusion, this project demonstrates the significant impact of graph-based AI models, particularly heterogeneous graph neural networks (HGNNs), on job-skill recommendation systems. By leveraging HGNNs, we capture complex, multi-faceted relationships within user-skill-job interactions, which traditional recommendation approaches often overlook. This graph-based approach enables a more nuanced understanding of user preferences and job requirements, resulting in recommendations that better align with individual user skills and roles. Our experiments showed that the heterogeneous model outperformed both matrix factorization and homogeneous graph models, achieving an MRR of 0.58 on the test set. This high score underscores the effectiveness of heterogeneous modeling in delivering personalized recommendations, especially in cases where accurate job-to-skill matching is critical.

The success of the HGNN model reveals potential for broader applications in recommendation systems beyond job markets. By focusing on message passing across distinct types of nodes and edges, the HGNN framework provides a versatile template that could be adapted to various domains, such as e-learning or talent acquisition platforms, where personalized recommendations are also in demand. Further optimization, such as adding more personalized pathways or exploring additional meta-paths, could make these systems even more robust and adaptable. Future research could explore extending this approach with real-time data, enabling more dynamic and up-to-date recommendations that continuously improve as new information is introduced. Overall, the HGNN approach highlights a promising direction for making AI-driven recommendations more contextually aware and effective across diverse, multi-entity datasets.

9 FUTURE WORK

Future research could explore introducing edge-level attention and to investigate real-time job updates for dynamic recommendation adjustments. Adding contextual data, such as company size or industry type, may also enhance recommendation precision.

10 Code

- User-based Collaborative Filtering Notebook
- GAT and HGNN Model Implementation

References

- [1] Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W.L., and Leskovec, J. (2018, July). Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 974-983.
- [2] Liu, P., Wei, H., Hou, X., Shen, J., He, S., Shen, K.Q., Chen, Z., Borisjuk, F., Hewlett, D., Wu, L., and Veeraraghavan, S. (2024). LinkSAGE: Optimizing Job Matching Using Graph Neural Networks. *arXiv preprint arXiv:2402.13430*.
- [3] Wang, X., Bo, D., Shi, C., Fan, S., Ye, Y., and Philip, S.Y. (2022). A survey on heterogeneous graph embedding: methods, techniques, applications and sources. *IEEE Transactions on Big Data*, 9(2), pp. 415-436.
- [4] Liu, H., Kong, X., Bai, X., Wang, W., Bekele, T.M., and Xia, F. (2015). Context-based collaborative filtering for citation recommendation. *IEEE Access*, 3, pp. 1695-1703.

- [5] Thorat, P.B., Goudar, R.M., and Barve, S. (2015). Survey on collaborative filtering, content-based filtering, and hybrid recommendation system. *International Journal of Computer Applications*, 110(4), pp. 31-36.
- [6] Labonne, M. (2023). *Hands-On Graph Neural Networks Using Python: Practical techniques and architectures for building powerful graph and deep learning apps with PyTorch*. Packt Publishing Ltd.
- [7] Topps, D., et al. (2023). Exploring Heterogeneous Graphs in Healthcare for Personalized Decision Making. *Journal of Healthcare AI*, 15(3), pp. 125-139.
- [8] Liu, L., Zhang, W., Liu, J., Shi, W., and Huang, Y. (2021, July). Learning multi-graph neural network for data-driven job skill prediction. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pp. 1-8. IEEE.
- [9] Jain, U., Jain, D., and Varshney, A.R. (2023). A Deep Learning Approach to Job Recommendation Analysis with NLP. *International Journal of Innovative Science and Research Technology*, 8(11), pp. 586-590.
- [10] Nigam, A., Roy, A., Singh, H., and Waila, H. (2019, December). Job recommendation through progression of job selection. In *2019 IEEE 6th International Conference on Cloud Computing and Intelligence Systems (CCIS)*, pp. 212-216. IEEE.